GitHub Research Audit: Deep Third Cumulants

Random ReLU MLPs | Round 3 follow-up | 21 September 2026

The useful finding is a correction to the object being optimized. I found no verified public implementation achieving both ≤1.2 u per source-layer and 2% query error. However, the actual ARC and V29 code carries a covariance-path contribution omitted from Round 3's “two propagator legs per source” description. This invalidates applying my previous 1.193-u estimate to the complete V29 chain.

The computational problem

Derived from the supplied target. At n=1024 and L=16, the task is to preserve the realized, weight-dependent third-cumulant fluctuations at every required nonlinear readout, while cutting roughly 100 u from a 260-u estimator. The target is approximately 1.6×10^{-8} final-mean MSE; B=1024 u and 1 u=2^{31} FLOPs. Low-rank propagator legs do not imply cheap Hadamard contractions. Your negative measurements remain constraints.

Code finding: from_dstensor is one summand

Code-verified [S1]. ARC first constructs *pK_111*, containing transported old K3 and newly born covariance-path terms. Only afterward does it add the repeated-index correction:

```
K_out[3] = pK_111 + FactoredTensor.from_dstensor(K_ds[3])
```

For the simple K3 update, write a=diag(w1) C_{off}, with columns a_j. The new path contribution is

$$K_{\text{path}} = \text{Sym}\sum_j (3a_j)\otimes e_j \otimes (w_{2,j}a_j).$$

Derived. This contains all-distinct indices at birth. A tensor $\text{Sym}\sum_j e_j\otimes e_j\otimes m_j$ has none. The latter is the residual correction, not the complete birth. With n=3, $a=11^T-I$ and w2=1, the path tensor has $K_{123}=3$, while every repeated-index correction has $K_{123}=0$.

The original first-round description included the missing AAP family. The inconsistency concerns the later identification of the MPP family with the entire reference source. The research bottleneck is real; the simplified source identification is not exact.

Evidence labels: CODE-VERIFIED = inspected public implementation; DERIVED = algebra or arithmetic; MEASURED-HERE = executed diagnostic; CONJECTURE = unverified inference. Public-leader and full-network accuracy figures are supplied by you, not independently reproduced here.

What V29 actually transports and contracts

Code-verified [S2]. Put P=T and A=T a_b. Ignoring only the separately implemented thin feedback terms, each birth contributes

$$D_{21} = 2(A \circ P \circ w_2)A^I + (A \circ A \circ w_2)P^I + \tfrac{1}{3}(P \circ P)M^T + \tfrac{2}{3}(M \circ P)P^T.$$

The first two terms are the AAP covariance-path family. The last two are the MPP repeated-index family. V29 already avoids transporting an independent dense M:

$$M = P\mathrm{diag}(s) + 3A\mathrm{diag}(e) + ZL^I.$$

Its two fused left factors contract against A^T and P^T. Thus the dense state is two n×n legs per young birth, plus thin legs and scalar vectors: 8 MiB for the two float32 legs. Updates are A←BA and P←BP. Two transports plus two readouts cost $4n^2(2n-1)$ FLOPs, or 3.998 u classically, approximately 2.04–2.05 u with the stated Strassen implementation. Old-basis pricing differs.

Correction to my previous PDF. The conditional 1.193-u calculation concerned a diagonal-plus-rank approximation of the lone birth matrix in an MPP-only model. In actual V29, M is already decomposed using A, P and a thin residual. Compressing M does not remove the covariance-path A leg, its transport or its readout. I withdraw that calculation as a cost proposal for the full estimator.

An executed structural falsifier

Measured here. I compared the path tensor with the tensor reconstructed from its exact repeated-index birth slices, then mixed both by the same random He layer. Frozen zero-mean Gaussian Wick gains were used. No learning or Monte Carlo truth was involved.

Diagnostic	Observed result
n=8: omitted tensor's birth D21, max |entry|	1.78×10^{-15}
n=8: dense tensor vs factored transported slice	$\le 2.56\times10^{-13}$
n=1024: relative transported D21 error	92.33%
n=1024: relative transported diagonal error	80.81%
n=1024: odd-parity residual under B → −B	0

Seven classical products cost 6.997 u; the additional parity check costs 1.999 u. Including pointwise operations and diagnostics, <9.1 u. Runtime here was 0.60 s with one BLAS thread. This is a two-layer structural test, **not** a 92% error estimate for the full 16-layer V29 chain. Appendix B supplies reproducible code.

Assessment of the four requested routes

Derived - shared squared propagators. For a row z of a shared left factor and a column x of a birth factor,

$$(z^T x)^2 = \sum_a z_a^2 x_a^2 + 2 \sum_{a<b} z_a z_b x_a x_b.$$

Sharing only z∘z loses the cross terms. An exact symmetric-pair lift has d=n(n+1)/2=524,800 coordinates. One n×d float32 array occupies 2.002 GiB; multiplying it by a d×n array costs $n^2(2d-1)/2^{31}$=512.500 u per readout. With inner rank 384, d=73,920 and the corresponding product still costs 72.187 u. These are direct lift costs, not universal lower bounds.

Derived - Gaussian column chaos. Conditional on prior-layer quantities and fixed gains, with standard Gaussian row z, Q=diag(w1)X and $\sigma^2=2/n$,

$$T_{ij}^2 = \sigma^2 \|Q_{:j}\|^2 + \sigma^2 \sum_{a,b} (z_a z_b - \delta_{ab}) Q_{aj} Q_{bj}.$$

This is exactly degree-two chaos; it is not a small coefficient state. Multiplication by the third transported leg produces cubic information. Keeping the realized chaos preserves oddness, but its coefficients return the same mixed-index core. Your trace/Hermite/sketch failures already reject the tested cheap truncations. A basis expansion alone supplies no additional saving.

Derived using your favorable pooled-core ledger. At r=250, the sum of slice evaluation, one birth join and one basis rotation is

$$\frac{nr^3 + 3r^4}{2^{31}} = 12.91 \text{ u/layer},$$

before other chain work. This exceeds 8 u/layer even under the stated optimistic counting. It does not become an $O(n^2)$ state by changing the name of the coefficients.

Already falsified - backward response. The source/query roles exchange, but mixed-index content survives in the adjoint. Your adjoint measurement closes that implementation. I found no public contraction schedule demonstrating a smaller complete bill on these tensors.

Assessment - kinetic or turbulence closures. A covariance of a quenched random tensor describes uncertainty about its realized odd part; it does not identify that part. Exact projected dynamics instead retain memory of unresolved components. A finite-memory closure would need new predictive evidence beyond your slice-regression and pooled-state failures. I found none at the required accuracy and cost.

The 2.05-u figure is an implementation cost, **not a proved arithmetic lower bound**. The search therefore supports “no demonstrated shortcut,” not “every possible shortcut is impossible.”

What the GitHub search added

Search scope. The exploratory sweep submitted 200 web queries, followed by targeted verification. Nine retained native GitHub repository searches returned 56 distinct repository candidates. Those counts are not counts of implementations read or validated, and do not constitute an exhaustive search of GitHub. Appendix C provides a reproducible selection of queries.

Code-verified [S3–S6]. TensorLy's memory-saving MTTKRP iterates over rank components; pyttb's sumtensor iterates over constituent tensors; opt_einsum caches identical intermediates. These help storage or scheduling, but none gives a source-independent state for this readout. ARC itself includes a symbolic whole-network contraction implementation; its documentation identifies it as a deprecated expansion and warns of impractical constants. This is relevant existing infrastructure, not a certified 133–165-u competitor.

Code-verified [S7]. AlphaTensor's released *real-arithmetic* 4×4 factorization has shape (3,16,49): 49 scalar multiplications, equal to two Strassen levels. This specific release entry does not halve the bill. Modulo-two records cannot simply be applied to float32 tensors. I am not claiming a lower bound on other real bilinear algorithms.

Potentially relevant, not a proposed solution [S8]. The 2026 AlternatingCrossInterpolation implementation avoids explicitly expanding TT Hadamard ranks to χ^2 when the output is compressible. Its state is TT cores; local updates contract interpolation frames and refactor a two-site block. It still requires a small output bond dimension. Your pooled three-mode TT result rejects that partition. A different binary/quantics partition was not measured, but no cheap, query-accurate transport under dense random W has been established. Pricing only the Hadamard sweep would omit the hard part.

Other nearby repositories concern low-rank covariance/Riccati projection, symmetric contractions of materialized tensors, marginal skewness, or Phase 1 sampling at 256×32. They do not supply the missing 1024×16 quenched cross-cumulant mechanism. Appendix A distinguishes inspected code from documentation-only leads.

What this says about the leaders

Unknown. I did not find a disclosed leading Phase 2 implementation that explains the gap. The numerical match between their bill and the young-tier bill does not prove what they carry. A 1.3× raw-accuracy margin likewise does not prove an additional cumulant order. V29 already has a cheap K4 regeneration channel [S2]. There is no verified new K4 implementation to price here.

I cannot name a demonstrated approximation that halves the complete bill while losing <1.3× in MSE. Recommending source deletion, hub screening or closure retuning would contradict your measurements.

The next measurement worth doing

Measure-this: audit complete-source query equivalence. Before optimizing the simplified MPP model, instrument the actual V29 run at layers 6, 10 and 14. From its cached legs, split each old-source readout into the covariance-path AAP contribution and the remaining MPP-plus-thin contribution. Keep the full chain's states and births fixed for this diagnostic.

$$D^{\text{path}}_{21,b} = 2(A_b \circ P_b \circ w_{2,b})A_b^T + (A_b \circ A_b \circ w_{2,b})P_b^T.$$

Accumulate the path contribution over old births, then compare the simplified-model readout with the **complete** reference. Report total D21 relative Frobenius error, diagonal error, and the fluctuations after using identical trace subtraction. Also report the norm and cross-inner-product of AAP and MPP; cancellation can make component norms misleading.

State and update. This is a diagnostic, not a new propagation scheme. Reuse existing A/P/w2; stream two n×n accumulators, about 8 MiB float32, plus product workspace. There is no fitted state or new recursive closure.

FLOPs. Isolating AAP adds two $n^2(2n-1)$ products per selected source, less than 2.01 u including pointwise work. For at most 30 old-source/checkpoint cases, reserve <62 u. With one existing V29 run, the total is <322 u; with stored legs it is only the diagnostic. No fresh 10^9-sample truth is needed. This should fit under an hour on a CPU that already runs your baseline; verify runtime locally.

First falsifier. Any complete-query error above 2% at a checkpoint rejects treating the MPP-only object as the full source for that network. If it passes, that is merely permission to test a complete recomputed chain, not proof of final-mean accuracy. The executed 92.33% structural test makes success unlikely, but does not replace this network-specific measurement.

Why this matters. It prevents spending another round optimizing a tensor that the estimator does not actually propagate. It also makes a proposed 1.2-u algorithm reviewable: its timed operation list must reproduce both AAP and MPP, thin corrections, joins and basis changes.

Decision. Retain V29 as the verified baseline. Treat the earlier diagonal-plus-low-rank birth shortcut as unvalidated for the actual chain. The useful product of this search is the corrected algebra and reproducible falsifier. A budget-breaking representation, and the leaders' mechanism, remain unresolved.

Appendix A / Source inspection inventory

These entries identify what was actually read. Downloading a repository is not counted as validating its algorithm. None of these external packages was run as a replacement for the full V29 chain.

Repository / inspected material	Finding for this problem
alignment-research-center/mlp_cumulant_propagation factor_k3.py; factor_k4.py; symb/README.md	Full birth includes covariance paths. Symbolic contraction infrastructure exists; no demonstrated target-cost solver.
504aldo/whest-p2-cumulant-k3 estimator_v29.py; lean code; findings/docs	The supplied baseline, not an independent breakthrough. M compression and readout fusion are already implemented.
AIcrowd/flopscope _flops.py; registry; matmul accounting	Confirms $n^2(2n-1)$ convention. Fast kernels alone do not change analytic charges.
tensorly/tensorly tenalg/core_tenalg/mttkrp.py	Memory-saving rank loop. No collapse of quenched third-order information.
sandialabs/pyttb sumtensor.py	Implicit sum; mttkrp explicitly loops over parts.
dgasmith/opt_einsum sharing.py	Memoizes repeated operations. Does not establish a new algebraic identity.
tensor4all/AlternatingCrossInterpolation.jl README and src/elementwise.jl	Actual 2026 Hadamard-TT algorithm; needs compressible TT output and affordable transport.
google-deepmind/alphatensor README and factorizations_r.npz	Inspected real 4×4 factorization: rank 49. Not the often-cited finite-field improvement.
Oishi1029/arc-whestbench-2026 README/write-up	Documentation: Phase 1, 256×32, sampling family. Wrong shape and a closed mechanism here.
aarushitandon0/white-box-estimation-arc README	Documentation: Gaussian covariance plus diagonal skewness; no required off-slice K3 state.
solomonik/sym_preserving_algs README	Documentation: symmetric matrix/tensor contractions. Already excluded by the supplied factored-leg setting.
marc-h-lambert/Riccati-PPCA README	Documentation: covariance projection. Does not supply realized odd-order closure.

Primary sources and version anchors

The two central implementation claims are pinned to commits. Other links identify the files inspected during this search; their main branches may change. Links are clickable.

[S1] ARC factored K3 source; commit 93d091a4c26c042bfffa28f2e76a81bc0aba94bb, from_dstensor and factored_nonlin_kprop_k3.

https://github.com/alignment-research-center/mlp_cumulant_propagation/blob/93d091a4c26c042bfffa28f2e76a81bc0aba94bb/src/mlp_kprop/factor_k3.py

[S2] V29 source; commit 18c17e2d7a9aeacd399cfc2c6b571e4e16dbfb45, file header and _dslices.

https://github.com/504aldo/whest-p2-cumulant-k3/blob/18c17e2d7a9aeacd399cfc2c6b571e4e16dbfb45/estimators/estimator_v29.py

[S3] TensorLy: explicit and memory-saving MTTKRP implementations.

https://github.com/tensorly/tensorly/blob/main/tensorly/tenalg/core_tenalg/mttkrp.py

[S4] PyTTB: sumtensor.mttkrp implementation.

https://github.com/sandialabs/pyttb/blob/main/pyttb/sumtensor.py

[S5] opt_einsum: shared-intermediate cache implementation.

https://github.com/dgasmith/opt_einsum/blob/main/opt_einsum/sharing.py

[S6] ARC symbolic cumulant-propagation implementation notes.

https://github.com/alignment-research-center/mlp_cumulant_propagation/blob/93d091a4c26c042bfffa28f2e76a81bc0aba94bb/src/mlp_kprop/symb/README.md

[S7] AlphaTensor: real and finite-field factorizations are separate assets.

https://github.com/google-deepmind/alphatensor/tree/main/algorithms

[S8] Alternating cross interpolation: local TT updates and truncation controls.

https://github.com/tensor4all/AlternatingCrossInterpolation.jl/blob/main/src/elementwise.jl

[S9] Ritter (2026), Fast elementwise operations on tensor trains with alternating cross interpolation.

https://arxiv.org/abs/2604.00037

[S10] FLOP accounting implementation, matmul_cost.

https://github.com/AIcrowd/flopscope/blob/main/src/flopscope/_flops.py

[S11] V29 community write-up and negative-results inventory.

https://github.com/504aldo/whest-p2-cumulant-k3/blob/18c17e2d7a9aeacd399cfc2c6b571e4e16dbfb45/docs/community_post.md

Appendix B / Reproduce the width-1024 diagnostic

The RNG precheck consumption is retained so these numbers match the executed audit. All matrix work is float32; float64 is used only for diagnostic norms. Formula construction and readout cost <9.1 u including the parity check; random-number generation is not part of the proposed estimator.

```
# NumPy only. Structural diagnostic, not a final-mean benchmark.
import os
os.environ["OPENBLAS_NUM_THREADS"] = "1"
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["MKL_NUM_THREADS"] = "1"
import numpy as np
rg = np.random.default_rng(20260921)

# Match RNG consumption of the small exact-tensor precheck.
rg.normal(size=(8, 8)); rg.normal(size=8)
rg.normal(size=(8, 8))
n = 1024
s = np.float32(np.sqrt(2/n))
W0 = rg.standard_normal((n,n), dtype=np.float32) * s
W1 = rg.standard_normal((n,n), dtype=np.float32) * s
C = W0 @ W0.T
w2 = np.float32(1/np.sqrt(2*np.pi)) / np.sqrt(np.diag(C))
A0 = np.float32(.5) * C
np.fill_diagonal(A0, 0)

# Exact repeated-index slice of the path tensor at birth.
birth = (A0*A0) * w2[None, :]
M0 = np.float32(3) * birth.T
P = np.float32(.5) * W1
A = P @ A0
M = P @ M0

def path(A, P):
    return (np.float32(2)*((A*P*w2[None,:]) @ A.T)
            + (A*A*w2[None,:]) @ P.T)

full = path(A, P)
rep = ((P*P) @ M.T + np.float32(2)*(P*M) @ P.T)/3
def norm(X):
    return np.sqrt(np.sum(X.astype(np.float64)**2))
print(norm(full-rep)/norm(full))       # 0.92329569
print(norm(np.diag(full-rep))/
      norm(np.diag(full)))            # 0.80810083
print(norm(path(-A,-P)+full)/norm(full))  # 0
```

Independent n=8 validation also materialized the two order-three tensors, transported all three axes, and compared their exact slices with these formulas. Agreement was within 2.56×10^{-13} in float64. This verifies the algebra without claiming full-network accuracy.

Appendix C / Reproducible search record

The exploratory sweep completed 200 web-query submissions. It was supplemented by the nine native GitHub repository searches below and targeted source checks. Many narrow web queries were unindexed or returned irrelevant pages; such hits were discarded. No exhaustive GitHub coverage is claimed.

Native GitHub search (retained JSON responses)

Exact repository-search query	Total matches / inspected result entries
WhestBench	5 / 5
white-box estimation ARC	8 / 8
cumulant ReLU	1 / 1
Sketchlord	0 / 0
tensor cross interpolation	20 / 15
implicit MTTKRP	0 / 0
symmetric tensor contraction	2 / 2
low rank diagonal	14 / 14
Mori Zwanzig	14 / 14

The returned entries contain 56 distinct repository names. “Result entries” means search metadata, not a code review. The 20-match tensor-cross query retained its first 15 entries.

Representative web queries 1–14

This is a reproducible selection, not the complete 200-query execution log. The following strings reproduce the final 40-query segment of the sweep.

01. site:github.com "FactoredTensor" "from_dstensor"
02. site:github.com "factored_nonlin_kprop_k3"
03. site:github.com "whest" "phase 2" "cumulant"
04. site:github.com "V29" "cumulant"
05. site:github.com "whest" "133"
06. site:github.com "whest" "1.64"
07. site:github.com "whest" "cumulant" "augment"
08. site:github.com "flopscope" "Strassen" "k3"
09. site:github.com "AlternatingCrossInterpolation" "Hadamard"
10. site:github.com "tensor train" "Gaussian" "Hadamard"
11. site:github.com "quantics" "random matrix"
12. site:github.com "tensor cross interpolation" "cumulant"
13. site:github.com "Mori" "Zwanzig" "cumulants"
14. site:github.com "turbulence" "third cumulant"

Representative web queries 15–40

15. site:github.com "CE3" "cumulant"
16. site:github.com "DIA" "closure" "turbulence"
17. GitHub ARC White Box Estimation phase 2 cumulant estimator 2026
18. GitHub whestbench-cumulant-propagation ascender1729
19. GitHub 504aldo whest-p2-cumulant-k3 findings
20. GitHub aarushitandon0 white-box-estimation-arc
21. GitHub tensor4all AlternatingCrossInterpolation.jl rank Hadamard
22. GitHub Sandia LearnNMZ Mori Zwanzig memory kernel
23. GitHub DeepMind alphatensor algorithms real arithmetic rank 4x4
24. GitHub Solomonik symmetry preserving tensor contraction algorithms
25. GitHub tensorly unfolding_dot_khatri_rao_memory
26. GitHub pyttb sumtensor mttkrp constituent tensors
27. GitHub opt_einsum sharing shared_intermediates
28. GitHub cotengra compressed contraction bond dimension
29. alternating cross interpolation tensor trains elementwise
30. cumulant propagation white box estimation
31. whestbench phase two estimator
32. matrix multiplication factorizations real arithmetic
33. tensor hypercontraction three index random tensor
34. Monarch matrices block diagonal butterfly projection
35. Mori Zwanzig reduced order closure memory
36. symmetric tensor contraction low rank bilinear
37. cumulant Edgeworth term selection neural network
38. tensor train Hadamard product rank truncation
39. Khatri Rao MTTKRP contraction optimization
40. white box estimation ARC competition submission

Queries 29–40 additionally used a github.com domain restriction. Named-function searches were supplemented with native repository search and direct public-source downloads when indexing failed. Search results and README claims were not treated as evidence of satisfying the 2% error or 133–165-u target.

Research stopping condition

The retrieved implementations either reproduce the known baseline, require the ranks already ruled out, address a different network shape, or do not provide the realized cross-cumulant state. No candidate passed a complete cost-and-accuracy validation. The positive finding is the code-backed source correction, with an executed falsifier.